CricGenius: Team Builder With Predictive AI

1 Introduction

1.1 Problem Understanding

The problem focuses on enhancing the team-building experience for Dream11 fantasy sports platform users, specifically for cricket.

We needed to build a robust machine learning (ML) model to predict player performance based on historical data, match conditions, and other relevant variables. Also, we need to provide detailed feature explainability to help users understand the rationale behind player recommendations. We must also design an intuitive user interface (UI) that simplifies team creation through interactive features like Generative AI-powered audio and video guides.

2 Background Study

We reviewed the following areas:

- Fantasy Sports Domain: Analyzed the Dream11 platform and user needs. Also looked into the Fantasy points distribution system.
- Existing Prediction Models: Went through different models like NodeGAM [1], regression models, tree-based models which are inherently explainable, and other than that, some Neural net models.
- Explainability Techniques: Explored different explainability techniques like SHAP [2] and Integrated Gradients for feature importance and transparency.

3 Data Processing

3.1 Input Data

The data for this project originates from CricSheet, containing ball-by-ball match performance statistics for players. The raw data is provided in JSON format, with each file representing a single match. Key details include:

- **Player Information**: Player name, team, and match date.
- **Match Context**: Venue, opposition team, and match format (Test, OD, or T20).
- **Performance Metrics**: Runs scored, wickets taken, balls faced, dot balls, boundaries hit/conceded, catches, and stumpings.

- **Fantasy Points**: Calculated from performance metrics, split into batting, bowling, and fielding contributions.

A README file accompanying the JSON data provides metadata such as match IDs, formats, and dates. These JSON files were processed into a structured dataset, including computed fantasy points and enriched features for machine learning.

3.2 Data Transformation

The raw JSON data was transformed into a structured format suitable for modeling:

- **Parsing JSON Files**:
  - Files were processed using the `cricketstats` library to extract player statistics.
  - Match format was standardized to Test, OD, or T20.
  - Fantasy points were computed following the rules from Dream11 fantasy sports platform
- **Augmenting Metadata**:
  - Match metadata (e.g., type, format, and opponent) was merged into the dataset from the README file.

3.3 Feature Engineering and Data Cleaning

Key feature engineering steps were implemented to enhance the dataset:

- **Cumulative Statistics**: Metrics like total runs, wickets, and dot balls, etc were calculated cumulatively for each player across their career to capture overall performance trends.
- **Rolling Averages**: We tested multiple k values for calculating rolling averages of key metrics (e.g., strike rate, economy rate, and boundary percentage) and observed their impact on model performance. Based on our analysis, we suggest $k = 15$ or $k = 20$, as these values effectively capture players' recent form while maintaining a balance between short-term and longer-term trends. Missing performance metrics (NaN values) were set to -1 (setting NaN values to -1 differentiates between players with zero scores and missing data.).
- **Derived Metrics**: Metrics like dismissal rate, boundary percentage, and bowling economy rate were computed to add depth to performance evaluation. Situations involving division by zero (e.g., economy rate or strike rate calculations) were handled by division with 1 instead of 0
- **Opponent and Venue-Specific Features**: Performance trends against specific opponents and at particular venues were analyzed and incorporated into the dataset.

3.4 Performance Optimization

To handle the large volume of data efficiently, the pipeline leverages parallel processing using Python's `concurrent.futures` library. Match files are processed concurrently across multiple threads, reducing runtime and enabling scalability for growing datasets.

3.5 Data Splitting and Export

- **Data Splitting by Date**:
  - **Training Dataset**: Includes matches up to and including June 30, 2024. Saved in the directory `../data/processed/train` with filenames formatted as `<k>_<match_type>.csv`
  - **Testing Dataset**: Includes matches after June 30, 2024. Saved in the directory `../data/processed/test` with filenames formatted as `<k>_<match_type>.csv`
- **Feature Engineering for Testing After Specific Date**:
  - Latest Match Data: The dataset was sorted by player names and match dates in descending order, retaining only each player's most recent match entry. The processed files are saved in the directory `../data/processed/test_after_10nov`, with filenames formatted as `<k>_<match_type>.csv`.

Implementation Details: The data splitting and export process was automated for multiple match formats (OD, T20, Test) using a Python script.

4 Our Approach

4.1 Multi-Layer Perceptron (MLP)

We have finalized a simple Multi-Layer Perceptron (MLP) model for this task, chosen for its balance between simplicity and effectiveness in capturing patterns relevant to predicting fantasy points. Other models, including Random Forest, Gradient Boosting, Bagging, NodeGAM, and Spline Transformer, were evaluated as baselines to benchmark performance.

4.2 Relevance of MLP in this Task

The relevance of using MLP in this context lies in its capacity to model non-linear relationships between the features (e.g., past performance metrics) and the target variable (fantasy points). This is crucial because the relationships in sports data are rarely linear. By leveraging the flexibility of MLP to handle non-linearity, the model can better capture complex patterns in the data, which is essential for making accurate predictions.

5 Explanation of Metrics

Mean Squared Error (MSE)

MSE measures the average of the squared differences between actual and predicted values:

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

Mean Absolute Error (MAE)

MAE calculates the average magnitude of errors in predictions, ignoring direction:

$$MAE = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

Mean Absolute Percentage Error (MAPE)

MAPE expresses accuracy as a percentage of errors relative to actual values:

$$MAPE = \frac{100}{n}\sum_{i=1}^{n}\left|\frac{y_i - \hat{y}_i}{y_i}\right|$$

Overlap Score

The Overlap Score measures the proportion of exact players between top 11 predicted players and top 11 actual players:

$$\text{Overlap Score} = \frac{1}{n}\sum_{i=1}^{n}\mathbb{I}(y_i = \hat{y}_i)$$

where $\mathbb{I}$ is 1 if $y_i = \hat{y}_i$, else 0.

6 Results and Discussion

ODI Results

Model	Mean Squared Error	Overlap Score
Random Forest Regressor	1783.50	5.50
Gradient Boosting	1693.55	6.50
Bagging Regressor	1736.57	6.20
AdaBoost Regressor	1783.88	6.42
Spline Transformer (XGBoost)	1678.99	6.50
MLP	1681.61	6.48

T20 Results

Model	Mean Squared Error	Overlap Score
Random Forest Regressor	1188.83	5.51
Gradient Boosting	1090.87	6.65
Bagging Regressor	1099.53	6.58
AdaBoost Regressor	1144.35	6.60
Spline Transformer (XGBoost)	1065.66	6.71
MLP	1064.27	6.69

Test Results

Model	Mean Squared Error	Overlap Score
Random Forest Regressor	3933.41	5.45
Gradient Boosting	3756.82	6.21
Bagging Regressor	3890.10	6.12
AdaBoost Regressor	3970.15	6.00
Spline Transformer (XGBoost)	3718.71	6.12
MLP	3712.50	6.26

MLP results

Format	MAE	MAPE	Overlap Score
OD	380.80	0.38	6.48
T20	272.31	0.30	6.69
Test	538.55	0.296	6.26

7 Explainability and Insights

7.1 Introduction

This section outlines our approach for prediction explanation system for fantasy cricket. The system explains the prediction process and generates explanations for players selected or excluded from fantasy teams.

7.2 Key Components and Workflow

7.2.1 Data Processing and Feature Extraction

The system tracks player features such as runs, wickets, boundary percentages, and dot balls. These features are analyzed over multiple matches to calculate a player's performance. Important features, identified through explainability methods, are used to generate relevant explanations.

7.2.2 Explainability Methods

The system uses the following explainability techniques:

- **Integrated Gradients**: Attributes model predictions to features by comparing the output with a baseline.
- **GradientSHAP**: Approximates Shapley values using a Monte Carlo approach to determine feature importance.

7.2.3 Generating Player Explanations

The system generates explanations by highlighting key features that contributed to the predicted fantasy score, differentiating between selected and unselected players:

- **Positive Explanations**: Highlight the strengths of players selected to the team.
- **Negative Explanations**: Emphasize why unselected players were not chosen, citing competition from others.

Explanations are provided in natural language, contextualizing player statistics (e.g., “Player X’s boundary count contributed to their high score”).

7.2.4 Generative Language Model Integration

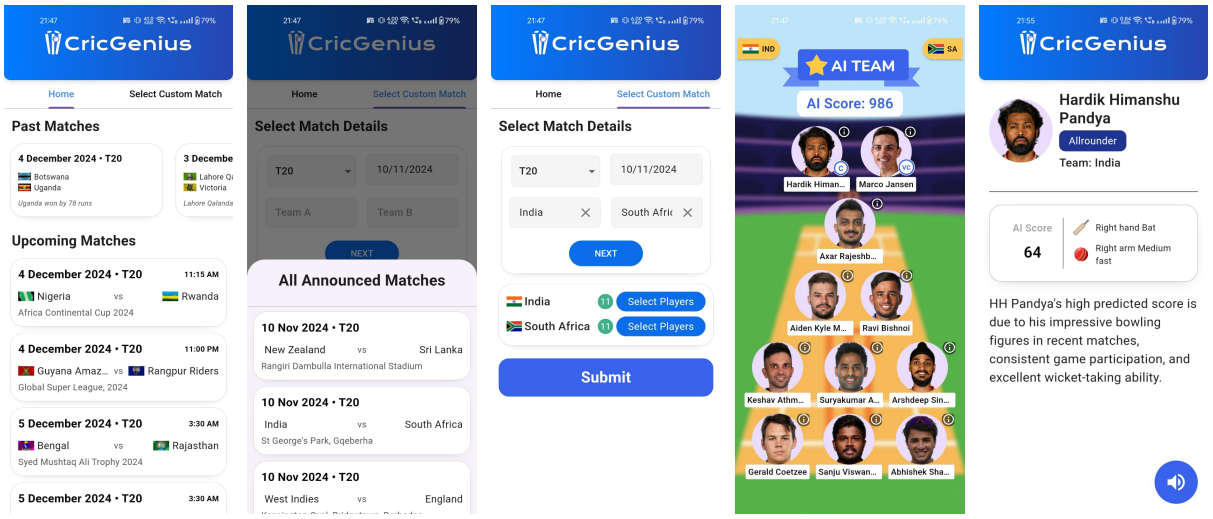
A generative model (e.g., Gemini-1.5-Flash) is used to turn feature attributions and predictions into human-readable explanations tailored to the cricket context, aiding user comprehension.

7.2.5 Backup and Redundant Explanations

The system stores backup explanations for structured insights, ensuring reliability in case of failures or further analysis.

8 UI Interfaces

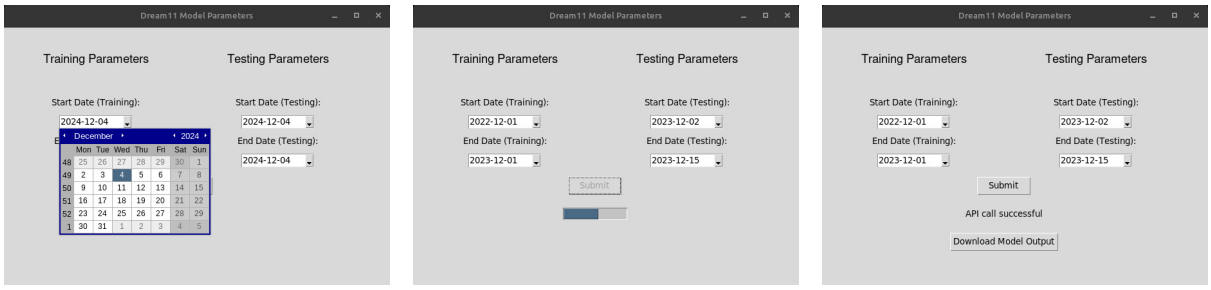
8.1 Player Card Recommendation, Explainability Interface



(a) Home screen (b) All matches (c) Chosen match (d) AI Team (e) Explanation

Figure 1: Product UI

8.2 Model UI



(a) Splash screen (b) API call (c) Download CSV

Figure 2: Model UI

9 Challenges

- We have given the ball-by-ball stats of each match. Extracting important features of players needed a lot of preprocessing.
- Needed to handle debut players as they didn't play any match before.
- Null and infinity values for features in the final datasets had to be handled carefully and intuitively.
- Choosing the best set of features that gave the best predictions was difficult and time-consuming.
- Choosing and training the best regression model from all the models we tried was also challenging.
- Issues in attaining good quality explainable texts that made sense in the game context.

10 Conclusion and Future Work

10.1 Summary

The report focuses on enhancing the Dream11 fantasy cricket platform using machine learning to predict player performance based on historical data and match conditions.

The data, sourced from CricSheet, was processed and enriched with features like cumulative statistics and rolling averages. A Multi-Layer Perceptron (MLP) model was selected for its ability to capture non-linear relationships in performance data. To ensure transparency, explainability techniques such as Integrated Gradients and GradientSHAP were employed to provide natural language explanations for player predictions. The UI includes features for team recommendations, match details, and player explanations.

Challenges faced during development included data preprocessing and handling debut players. Future work includes adding multi-language support, integrating with Dream11, and enhancing the user experience with Text-to-Speech features.

10.2 Recommendations for Improvement

- User Experience wise, to make everything more usable by users all over the country, we can include option to change language of the App and the Text-to-Speech feature
- We can connect this App with the Dream11 App by opening up an API from the main app to this app and including a feature to swap players in the recommended AI team by user-chosen players so that users will be able to send a custom team directly to the Dream11 App.

References

[1] Chun-Hao Chang, Rich Caruana, and Anna Goldenberg. Node-gam: Neural generalized additive model for interpretable deep learning, 2022.

[2] Scott Lundberg and Su-In Lee. A unified approach to interpreting model predictions, 2017.